

**TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN
A DISTANCIA - UTN**

Programación I

Trabajo Práctico Integrador
Sistema de Gestión de Países

Grupo nº 209 – Integrantes:

Barrios, Nair Julieta

Rossini, Máximo

15 de Junio de 2026

Índice

Contenido

Índice	2
1. Introducción	4
2. Objetivo del proyecto.....	4
3. Marco teórico.....	4
3.1. Pensamiento computacional y algoritmos.....	4
3.2. Estructuras de datos: listas y diccionarios	4
3.3. Funciones y modularización	5
3.4. Archivos CSV y persistencia de datos	5
3.5. Validaciones y manejo de errores	5
3.6. Algoritmos de ordenamiento.....	5
4. Descripción general del sistema	5
5. Dataset utilizado.....	6
6. Decisiones técnicas y arquitectura.....	6
6.1. Estructura del proyecto.....	6
6.2. Diseño modular del código	7
6.3. Diagrama de flujo de operaciones principales	7
7. Funcionalidades implementadas.....	8
7.1. Mostrar países	8
7.2. Agregar país	8
7.3. Actualizar país	8
7.4. Buscar país por nombre	8
7.5. Filtrar países	8
7.6. Ordenar países	8
7.7. Mostrar estadísticas.....	8
8. Validaciones y manejo de errores implementados	8
8.1. Validaciones de entrada	8
8.2. Manejo de errores con archivos	8
9. Ordenamiento aplicado	9
10. Pruebas y capturas de ejecución	9
10.1. Captura del menú principal y listado de países	10
10.2. Captura de búsqueda	10
10.3. Captura de estadísticas.....	11

11. Herramientas utilizadas.....	11
12. Dificultades encontradas y resolución	11
13. Conclusiones.....	12
14. Bibliografía / Webgrafía.....	12
15. Links del proyecto y anexos.....	12
15.1. Repositorio del proyecto.....	12
15.2. Video explicativo.....	12

1. Introducción

El presente informe corresponde al Trabajo Práctico Integrador de la materia Programación I. El proyecto desarrollado consiste en una aplicación de consola realizada en Python que permite gestionar información de países almacenada en un archivo CSV.

La propuesta integra contenidos trabajados durante la cursada, tales como variables, estructuras condicionales y repetitivas, listas, diccionarios, funciones, archivos, manejo de errores, algoritmos de ordenamiento y modularización del código. El sistema permite consultar, agregar, actualizar, buscar, filtrar, ordenar y analizar datos de países, simulando una herramienta simple de gestión de información persistente.

2. Objetivo del proyecto

El objetivo principal del proyecto es desarrollar una aplicación funcional en Python que permita gestionar un conjunto de datos de países mediante un menú interactivo por consola.

- Aplicar estructuras de datos como listas y diccionarios.
- Leer y escribir información en archivos CSV.
- Implementar funciones con responsabilidades específicas.
- Validar datos ingresados por el usuario.
- Manejar errores frecuentes relacionados con archivos y datos numéricos.
- Ordenar información mediante un algoritmo visto en la materia.
- Presentar estadísticas generales a partir del dataset cargado.

3. Marco teórico

El desarrollo del sistema se apoya en distintos conceptos centrales de Programación I. A continuación, se explican los contenidos aplicados y su relación con el proyecto.

3.1. Pensamiento computacional y algoritmos

Programar implica resolver problemas a través de instrucciones ordenadas. Para ello, se aplica pensamiento computacional, que permite descomponer un problema complejo en partes más pequeñas, reconocer patrones y diseñar algoritmos. En este proyecto, el problema general de gestionar países se dividió en operaciones concretas: cargar datos, mostrar información, agregar registros, actualizar datos, filtrar, ordenar y calcular estadísticas.

Un algoritmo es una secuencia finita y ordenada de pasos para resolver una tarea. El menú principal del sistema funciona como guía del flujo general del programa, mientras que cada función representa un algoritmo específico para una operación determinada.

3.2. Estructuras de datos: listas y diccionarios

Una lista es una estructura que permite almacenar una colección ordenada de elementos. En el proyecto, la lista principal contiene todos los países cargados desde el archivo CSV. Esta estructura permite recorrer los datos con ciclos, agregar nuevos elementos y aplicar filtros u ordenamientos.

Cada país se representa mediante un diccionario. Un diccionario almacena información en pares clave-valor; por ejemplo, las claves utilizadas son nombre, poblacion, superficie y continente. Esta elección

facilita el acceso directo a cada dato mediante su clave, evitando depender de posiciones numéricas difíciles de recordar.

3.3. Funciones y modularización

Una función es un bloque de código reutilizable que realiza una tarea específica. En este proyecto se utilizaron funciones para organizar el programa, evitar repetición de código y facilitar el mantenimiento. Cada función cumple una responsabilidad concreta, por ejemplo leer el archivo CSV, guardar cambios, mostrar países, filtrar por continente o calcular estadísticas.

La modularización se aplicó en dos niveles: mediante funciones y mediante separación en archivos. Esta decisión mejora la claridad del proyecto, ya que el archivo `main.py` se ocupa del menú, `archivo_paises.py` de la lectura y escritura del CSV, y `gestion_paises.py` de la lógica principal del sistema.

3.4. Archivos CSV y persistencia de datos

Un archivo CSV permite almacenar datos en formato tabular, donde cada línea representa una fila y cada valor separado por comas representa una celda. En este sistema, el archivo `paises.csv` funciona como una base de datos simple. Gracias a esto, los datos no se pierden al finalizar el programa.

Para trabajar con el CSV se utilizó el módulo `csv` de Python, específicamente `DictReader` para leer cada fila como diccionario y `DictWriter` para escribir nuevamente los datos respetando los encabezados.

3.5. Validaciones y manejo de errores

Las validaciones permiten controlar los datos ingresados por el usuario antes de utilizarlos. En el sistema se verifica que los textos no estén vacíos, que la población y la superficie sean números enteros positivos, y que los rangos ingresados sean coherentes.

El manejo de errores se realizó con bloques `try` y `except`. Esto evita que el programa se detenga abruptamente ante problemas previsibles, como que el archivo CSV no exista, que un dato numérico tenga formato incorrecto o que el archivo no pueda guardarse por estar abierto en otro programa.

3.6. Algoritmos de ordenamiento

Ordenar consiste en reorganizar elementos de una colección según un criterio. En el proyecto se implementó Bubble Sort mejorado, un algoritmo que compara elementos adyacentes y los intercambia si se encuentran en un orden incorrecto. La mejora consiste en detener el proceso si en una pasada completa no hubo intercambios, lo que indica que la lista ya está ordenada.

El algoritmo se adaptó para ordenar una lista de diccionarios, recibiendo como parámetro el campo por el cual se desea ordenar: nombre, población o superficie.

4. Descripción general del sistema

El sistema desarrollado se ejecuta desde la consola y presenta un menú principal con diferentes opciones. A partir de ese menú, el usuario puede elegir qué operación realizar sobre el conjunto de países.

Cada país se representa mediante un diccionario con cuatro campos principales: nombre, población, superficie y continente. Todos los países se almacenan dentro de una lista, lo que permite recorrerlos, filtrarlos, modificarlos y ordenarlos. La información se conserva en el archivo CSV `países.csv`.

```
--- Sistema de Gestión de Países ---
1. Mostrar países
2. Agregar país
3. Actualizar país
4. Buscar país por nombre
5. Filtrar países
6. Ordenar países
7. Mostrar estadísticas
8. Salir
```

5. Dataset utilizado

El dataset utilizado se encuentra en el archivo `países.csv`. El archivo está organizado en formato tabular: la primera fila contiene los encabezados y cada fila posterior representa un país.

Campo	Tipo de dato	Descripción	Ejemplo
nombre	Texto	Nombre del país.	Argentina
poblacion	Entero	Cantidad de habitantes.	45376763
superficie	Entero	Superficie en km².	2780400
continente	Texto	Continente al que pertenece.	América

Ejemplo de estructura del archivo CSV:

```
nombre,poblacion,superficie,continente
Argentina,45376763,2780400,América
Brasil,213993437,8515767,América
España,47450795,505990,Europa
China,1412000000,9596961,Asia
```

6. Decisiones técnicas y arquitectura

Para cumplir con los criterios de claridad, modularización y mantenimiento, se tomó la decisión de separar el sistema en módulos. Esta arquitectura permite distinguir claramente la interacción con el usuario, el acceso al archivo CSV y la lógica de gestión de países.

6.1. Estructura del proyecto

```
tpi_paises/
├── graphs
│   └── diagrama_flujo.pdf
├── tpi_barrios_rossini.pdf
├── main.py
├── archivo_paises.py
├── gestion_paises.py
├── países.csv
└── README.md
```

Archivo	Responsabilidad
diagrama_flujo.pdf	Contiene el diagrama de flujo del programa
tpi_barrios_rossini.pdf	Contiene el PDF obligatorio de presentación
main.py	Contiene el menú principal y controla la ejecución general del programa.
archivo_paises.py	Contiene las funciones que leen y guardan datos en el archivo CSV.
gestion_paises.py	Contiene la lógica de gestión: mostrar, agregar, actualizar, buscar, filtrar, ordenar y calcular estadísticas.
paises.csv	Archivo de datos donde se almacena la información de los países.
README.md	Documento de presentación del proyecto dentro del repositorio.

6.2. Diseño modular del código

El archivo main.py funciona como punto de entrada del sistema. Desde allí se carga la información inicial del CSV, se muestra el menú principal y se llama a las funciones correspondientes según la opción elegida por el usuario.

El archivo archivo_paises.py contiene las funciones relacionadas con la lectura y escritura del CSV: leer_paises(nombre_archivo) y guardar_paises(nombre_archivo, paises). El archivo gestion_paises.py contiene las funciones principales del sistema: mostrar, agregar, actualizar, buscar, filtrar, ordenar y calcular estadísticas.

6.3. Diagrama de flujo de operaciones principales

El siguiente diagrama resume el flujo principal del programa desde que se inicia hasta que el usuario decide salir.

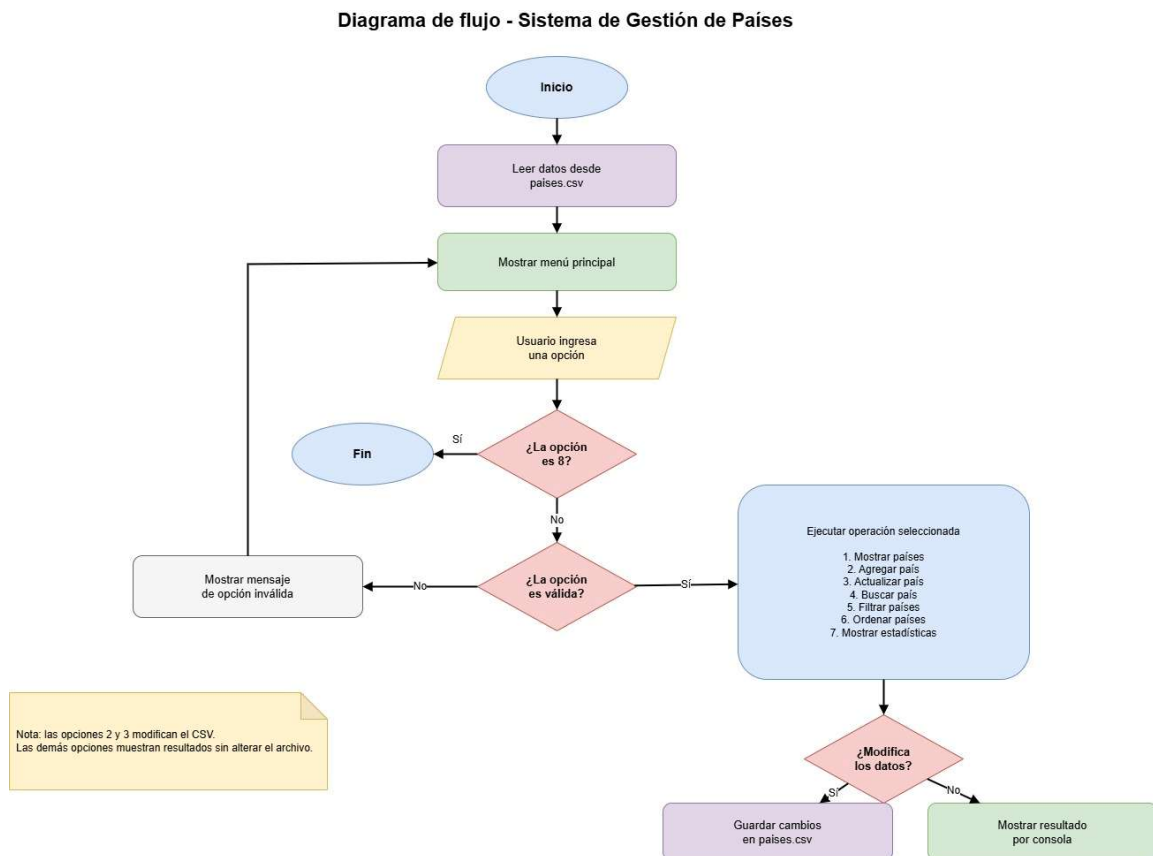


Figura 1. Diagrama de flujo general del Sistema de Gestión de Países.

7. Funcionalidades implementadas

7.1. Mostrar países

Permite visualizar todos los países cargados en el archivo CSV. Para cada país se muestra su nombre, población, superficie y continente.

7.2. Agregar país

Permite ingresar un nuevo país al sistema. El programa solicita nombre, población, superficie y continente. Luego agrega el nuevo país a la lista y guarda los cambios en el archivo CSV.

7.3. Actualizar país

Permite modificar la población y superficie de un país ya existente. La búsqueda se realiza por nombre exacto.

7.4. Buscar país por nombre

Permite buscar países por nombre completo o parcial. Para esto se recorre la lista de países y se agregan a una lista de resultados aquellos cuyo nombre contenga el texto ingresado.

7.5. Filtrar países

Permite filtrar por continente, por rango de población o por rango de superficie.

7.6. Ordenar países

Permite ordenar los países por nombre, población o superficie, de forma ascendente o descendente.

7.7. Mostrar estadísticas

Calcula país con mayor población, país con menor población, promedio de población, promedio de superficie y cantidad de países por continente.

8. Validaciones y manejo de errores implementados

El programa incorpora validaciones para evitar errores frecuentes durante la ejecución. Estas validaciones ayudan a que el sistema sea más robusto y brinde mensajes claros al usuario.

8.1. Validaciones de entrada

- Los campos de texto no pueden quedar vacíos.
- La población debe ser un número entero positivo.
- La superficie debe ser un número entero positivo.
- En los filtros por rango, el valor mínimo no puede ser mayor que el máximo.
- El menú principal y los submenús validan opciones incorrectas.

8.2. Manejo de errores con archivos

- `FileNotFoundException`: se muestra un mensaje si el archivo CSV no existe.
- `ValueError`: se muestra un mensaje si los datos numéricos del CSV tienen formato incorrecto.

- **PermissionError:** se muestra un mensaje si no se puede guardar el archivo, por ejemplo, si está abierto en otro programa.

9. Ordenamiento aplicado

Para ordenar los países se utilizó el algoritmo Bubble Sort mejorado. Este algoritmo compara elementos adyacentes y los intercambia si se encuentran en un orden incorrecto.

La versión mejorada utiliza una variable que permite detectar si en una pasada completa no hubo intercambios. En ese caso, significa que la lista ya está ordenada y no es necesario continuar recorriéndola.

En el proyecto, el algoritmo se adaptó para ordenar una lista de diccionarios. El campo por el cual se ordena se recibe como parámetro y puede ser nombre, población o superficie.

```
def ordenar_lista_paises(paises, campo, descendente):
    paises_ordenados = copiar_lista_paises(paises)
    n = len(paises_ordenados)

    for i in range(n - 1):
        hubo_intercambio = False
        # Comparación de países según el campo elegido
```

10. Pruebas y capturas de ejecución

Para verificar el correcto funcionamiento del sistema, se realizaron pruebas manuales desde la consola. A continuación se documentan casos de prueba y ejemplos visuales de ejecución.

Prueba	Acción realizada	Resultado esperado
1	Mostrar países	Se muestra el listado completo de países.
2	Agregar un país con datos válidos	El país se agrega y se guarda en el CSV.
3	Actualizar población y superficie	Los datos del país se modifican correctamente.
4	Buscar país por parte del nombre	Se muestran los países que coinciden con la búsqueda.
5	Filtrar por continente	Se muestran solo países del continente ingresado.
6	Ordenar por población descendente	Los países se muestran de mayor a menor población.
7	Mostrar estadísticas	Se muestran máximos, mínimos, promedios y conteo por continente.

10.1. Captura del menú principal y listado de países

```
PROBLEMS  OUTPUT  TERMINAL  PORTS

--- Sistema de Gestión de Países ---
1. Mostrar países
2. Agregar país
3. Actualizar país
4. Buscar país por nombre
5. Filtrar países
6. Ordenar países
7. Mostrar estadísticas
8. Salir
Ingrese una opción: 1

Listado de países:
Nombre: Argentina | Población: 45376763 | Superficie: 2780400 km² | Continente: América
Nombre: Brasil | Población: 213993437 | Superficie: 8515767 km² | Continente: América
Nombre: Chile | Población: 19493184 | Superficie: 756102 km² | Continente: América
Nombre: Uruguay | Población: 3426260 | Superficie: 176215 km² | Continente: América
Nombre: Colombia | Población: 51516562 | Superficie: 1141748 km² | Continente: América
Nombre: Mexico | Población: 126014024 | Superficie: 1964375 km² | Continente: América
Nombre: Alemania | Población: 82140700 | Superficie: 357022 km² | Continente: Europa
```

Figura 2. Ejemplo visual de ejecución: menú principal y listado de países.

10.2. Captura de búsqueda

```
PROBLEMS  OUTPUT  TERMINAL  PORTS

1. Mostrar países
2. Agregar país
3. Actualizar país
4. Buscar país por nombre
5. Filtrar países
6. Ordenar países
7. Mostrar estadísticas
8. Salir
Ingrese una opción: 4

Buscar país por nombre
Ingrese el nombre o parte del nombre del país: Arg

Resultados encontrados:

Listado de países:
Nombre: Argentina | Población: 45376763 | Superficie: 2780400 km² | Continente: América
Nombre: Argelia | Población: 46810000 | Superficie: 2381741 km² | Continente: África
```

Figura 3. Ejemplo visual de ejecución: búsqueda parcial.

10.3. Captura de estadísticas

```
PROBLEMS  OUTPUT  TERMINAL  PORTS

4. Buscar país por nombre
5. Filtrar países
6. Ordenar países
7. Mostrar estadísticas
8. Salir
Ingrese una opción: 7

--- Estadísticas del dataset ---
País con mayor población: China (1412000000)
País con menor población: Perú (3400000)
Promedio de población: 217088278.61
Promedio de superficie: 2438628.94 km²

Cantidad de países por continente:
América: 7
Europa: 4
Asia: 3
Oceanía: 1
África: 3
```

Figura 4. Ejemplo visual de ejecución: salida de estadísticas generales.

11. Herramientas utilizadas

- Python 3 para el desarrollo de la aplicación.
- Visual Studio Code como editor de código.
- Git para el control de versiones.
- GitHub para alojar el repositorio del proyecto.
- Archivo CSV como sistema de almacenamiento simple.
- Módulo csv de Python para leer y escribir datos estructurados.

12. Dificultades encontradas y resolución

Durante el desarrollo del proyecto surgieron distintas dificultades técnicas. A continuación, se detallan los principales obstáculos y la forma en que fueron abordados.

Dificultad	Causa	Resolución
Lectura del CSV	Era necesario transformar los datos leídos como texto a tipos adecuados.	Se utilizó <code>csv.DictReader</code> y conversión explícita a <code>int</code> para población y superficie.
Persistencia de datos	Al agregar o actualizar países, los cambios debían conservarse al cerrar el programa.	Se implementó una función <code>guardar_paises</code> que reescribe el CSV con los datos actualizados.
Validación de entradas	El usuario podía ingresar campos vacíos o valores no numéricos.	Se crearon funciones de validación para textos no vacíos y enteros positivos.
Ordenamiento	El dataset está compuesto por diccionarios, no por valores simples.	Se adaptó Bubble Sort para comparar el valor de una clave dinámica recibida como parámetro.
Modularización	El código inicialmente podía quedar demasiado extenso en un solo archivo.	Se separó el proyecto en <code>main.py</code> , <code>archivo_paises.py</code> y <code>gestion_paises.py</code> .

13. Conclusiones

El desarrollo del Sistema de Gestión de Países permitió integrar los principales contenidos trabajados durante Programación I en un proyecto funcional y organizado.

A partir del uso de listas y diccionarios se logró representar y manipular información estructurada. Mediante archivos CSV se incorporó persistencia de datos, permitiendo que la información se conserve entre distintas ejecuciones del programa.

La división del código en funciones y archivos mejoró la claridad del proyecto, ya que cada parte del sistema cumple una responsabilidad específica. Esto facilita futuras modificaciones, correcciones o ampliaciones.

Como reflexión grupal, el trabajo permitió comprender la importancia de planificar la estructura del programa antes de escribir código, probar cada funcionalidad por separado y utilizar GitHub para mantener un historial ordenado del desarrollo.

14. Bibliografía / Webgrafía

Las fuentes consultadas para respaldar el marco teórico y las decisiones técnicas fueron las siguientes:

- Material de cátedra: Introducción a la Programación y Herramientas Básicas en Python.
- Material de cátedra: Estructuras condicionales, operadores relacionales y operadores lógicos.
- Material de cátedra: Estructuras de repetición.
- Material de cátedra: Listas, arreglos y algoritmos de ordenamiento.
- Material de cátedra: Funciones en Python, modularidad y reutilización de código.
- Material de cátedra: Diccionarios y estructuras de datos complejas.
- Material de cátedra: Manipulación de archivos, archivos CSV y manejo de errores.
- Python Software Foundation. Documentación oficial del módulo csv:
<https://docs.python.org/3/library/csv.html>
- Python Software Foundation. Tutorial oficial de estructuras de datos:
<https://docs.python.org/3/tutorial/datastructures.html>
- Python Software Foundation. Tutorial oficial de errores y excepciones:
<https://docs.python.org/3/tutorial/errors.html>
- Git Documentation: <https://git-scm.com/doc>
- GitHub Docs: <https://docs.github.com/>

15. Links del proyecto y anexos

15.1. Repositorio del proyecto

Repositorio GitHub: https://github.com/julietabarrios93/TPI_Programacion1_Barrios_Rossini

15.2. Video explicativo

Video de presentación: <https://www.youtube.com/watch?v=Bv28cu-r4Cw>